

Post-deploy smoke tests, pipeline gates, and rollback

Practical guide for a local Windows setup running Rancher Desktop Kubernetes (containerd), with the source repository in WSL Ubuntu, images published to GitLab Container Registry, and post-deploy validation plus automatic rollback to the previous image tag.

What this guide is: a “worked example” that documents the exact approach that ended up working, including the blockers that came up and how they were fixed.

What this guide is not: a list of optional architectures. Where options are mentioned, it is because they were explored during troubleshooting, and the reason they were not used is documented.

Acceptance criteria: (1) smoke tests exist and run automatically after deployment; (2) rollback exists, is documented, and restores the previous known-good image tag when smoke fails.

How to read this document

If the goal is simply to repeat the working setup, jump straight to **End-to-end runbook**. If something breaks, use **Troubleshooting**. For understanding why each step exists, read **Concepts** and **What was learned**.

Sections

- **Concepts**: smoke tests, gates, rollback, tags, registries, NodePort vs port-forward.
- **Environment**: the exact split between WSL, Windows, containerd, and GitLab registry.
- **What was learned**: the blockers encountered and fixes that worked.
- **Implementation**: build, deploy, expose, smoke, gate, rollback test.
- **Automation**: what “automatically post-deploy” means in a GitLab pipeline.
- **End-to-end runbook**: the shortest repeatable checklist.

Concepts and mental model

This implementation is easier to remember with one simple rule: **a deploy is not “done” until something proves the service is healthy.**

Smoke test

A smoke test is a minimal check that answers: “is the system alive enough to serve traffic?”. For a web service, that often means making a few HTTP requests and expecting HTTP 200.

In this setup, the smoke checks reuse the same endpoints already configured as Kubernetes health probes: `/actuator/health/liveness` and `/actuator/health/readiness`.

Pipeline gate

A pipeline gate is a “stop sign” in automation: if the gate fails, the pipeline must not claim success. Here, the smoke test step is the gate, and it runs immediately after deployment.

Rollback

Rollback means restoring the previous known-good version. In this guide, “known-good” is defined as: **the image that was running just before attempting the new deploy**. The gate captures that previous image reference (including its tag), and uses it if a rollback is needed.

Image tags and why avoiding `:latest` helps

Tags identify versions. Using unique tags like `smoke1` makes rollback unambiguous. Using `:latest` can make debugging and rollback confusing because the same tag might point to different content over time.

NodePort vs port-forward (why localhost did not work)

A common confusion is expecting in-cluster services to automatically show up on Windows `localhost`. Kubernetes does not do that by default.

NodePort opens a port on the Kubernetes node network. On desktop clusters, that node network can be isolated behind a VM or NAT, so the node IP may not be reachable from the host OS. That is why calls to `nodeIP:32000` timed out here.

Port-forward tunnels traffic through the Kubernetes API and creates a local listener on the current machine. That is why `kubectl port-forward` worked reliably for both the registry and the app.

Environment used in the working solution

- **Windows 11:** Rancher Desktop Kubernetes (containerd) and cluster networking.
- **WSL Ubuntu:** repository working directory, scripts folder, and the image build/push tool.
- **Podman in WSL:** builds the container image (multi-stage Dockerfile) and pushes to GitLab registry.
- **GitLab Container Registry:** source of truth for images pulled by Kubernetes (via imagePullSecrets).
- **Petclinic:** Deployment in namespace `petclinic` with actuator liveness/readiness endpoints enabled.

Why containerd mattered

Rancher Desktop uses containerd, which often means the classic Docker engine workflow is not guaranteed. Instead of fighting the runtime mismatch, the solution used Podman in WSL to build and push images to GitLab registry, then Kubernetes pulled images normally from the registry.

Where to place scripts

Create `scripts/` at the repository root (next to `app/`, `k8s/`, `ci/`). This makes paths consistent for local runs and CI.

```
PetClinic-InfraLab/  
  app/  
  ci/  
  docs/  
  infra/  
  k8s/  
  scripts/  
  .gitlab-ci.yml  
  README.md
```

Security note: do not include scripts that print secrets, tokens, or kubeconfig contents. Keep tokens in masked variables and do not echo them.

What was learned: blockers and fixes that worked

This section is intentionally narrative. It records the “why” behind the final approach, based on the actual problems encountered.

1) Local registry check failed on localhost:5000

A curl request to `http://localhost:5000/v2/_catalog` failed with “could not connect”. Windows checks confirmed that nothing was listening on port 5000 on the host.

```
curl http://localhost:5000/v2/_catalog
Test-NetConnection localhost -Port 5000
netstat -ano | findstr :5000
```

The important realization: the registry was running inside Kubernetes. That does not automatically bind to Windows localhost.

2) The registry existed, but NodePort was not reachable

Kubernetes showed a registry Service of type NodePort (port 5000 mapped to a node port such as 32000). However, attempts to reach `nodeIP:32000` timed out. This is a known “desktop cluster” behavior when the node IP is not routable from the host.

```
kubectl get nodes -o wide
kubectl -n registry get svc
curl http://<node-ip>:32000/v2/_catalog # timed out in this setup
```

Fix that worked: port-forward

Port-forward exposed the in-cluster registry on Windows localhost. This validated that the registry itself was fine; only host routing was the issue.

```
kubectl -n registry port-forward svc/docker-registry 5000:5000
curl http://localhost:5000/v2/_catalog
```

3) Choosing the registry strategy: local registry vs GitLab registry

Two variants were discussed: pushing images to the in-cluster registry (local variant) or using GitLab Container Registry (GitLab variant). The GitLab variant was chosen because it avoids the NodePort reachability problem and matches the existing Deployment’s `imagePullSecrets` approach.

The key difference: local registry is great for isolated local development, but GitLab registry is the “real pipeline” source of truth and is reachable from everywhere.

4) Repo in WSL, kubectl on Windows

The repository lived inside WSL. Running commands from Windows PowerShell caused path mismatch issues (e.g., build tools searching for Dockerfile in the wrong directory). The fix was to keep build scripts in WSL and enable kubectl access from WSL by reusing the Windows kubeconfig.

```
export KUBECONFIG=/mnt/c/Users/<windows-user>/.kube/config
# or:
mkdir -p ~/.kube && cp /mnt/c/Users/<windows-user>/.kube/config ~/.kube/config
kubectl get nodes
```

Both methods are valid. Exporting KUBECONFIG stays “single source of truth”; copying is more Linux-native but needs re-sync if the Windows kubeconfig changes.

5) Build tool choice: Podman in WSL

Because the repo lived in WSL and Podman was available there, the successful workflow used Podman to build and push images. The Dockerfile was multi-stage and provided a target that builds the JAR inside the container image.

Implementation steps (the working path)

This section is written as a sequence of actions. The commands are the same ones used during the successful run.

Step 1 - Build and push the image (WSL, Podman)

Run commands from the repository root in WSL. Use a unique tag for each build to make rollback predictable.

```
TAG=smoke1
podman login gitlab-registry.endava.com
podman build --target runtime -t gitlab-registry.endava.com/claudiu.sonel/petclinic-infralab:$TAG .
podman push gitlab-registry.endava.com/claudiu.sonel/petclinic-infralab:$TAG
```

Security: the login command will prompt for credentials; avoid hardcoding tokens in commands or storing them in the repository.

Step 2 - Deploy the tag to Kubernetes (kubectl)

Update the Deployment image and wait for rollout completion. This is the “deploy” part of the gate.

```
kubectl -n petclinic set image deploy/petclinic \
  petclinic=gitlab-registry.endava.com/claudiu.sonel/petclinic-infralab:$TAG
kubectl -n petclinic rollout status deploy/petclinic --timeout=10m
```

What this does behind the scenes

- Edits the Deployment Pod template image for the container named `petclinic`.
- Creates a new ReplicaSet, starts new Pods, and only considers them “Ready” when readinessProbe passes.
- Replaces the old Pods once the new ones are ready.

If something goes wrong

The fastest signal is Pod events. Image pull failures or readiness failures show up there.

```
kubectl -n petclinic get pods
kubectl -n petclinic describe pod <pod-name>
```

If image pull fails, verify the imagePullSecret exists and is correct. Avoid printing secret data or tokens.

Step 3 - Expose the service locally for validation

During local testing, port-forward creates `http://localhost:8080` on the machine that runs `kubectl`. Leave the port-forward running while smoke tests execute.

```
kubectl -n petclinic get svc
kubectl -n petclinic port-forward svc/petclinic 8080:80 # or 8080:8080
```

Quick health check (manual)

```
curl -f http://localhost:8080/actuator/health/liveness
curl -f http://localhost:8080/actuator/health/readiness
```

Step 4 - Smoke test (script)

The smoke test script runs the same checks, with retries to handle startup time. Its exit code is what turns it into a gate: 0 means pass; non-zero means fail.

```
./scripts/smoke-test.sh http://localhost:8080
```

Step 5 - Deploy gate (smoke + rollback)

The deploy gate script wraps the whole workflow: capture the current image (previous known-good), deploy the new image, run smoke. If smoke fails, set the image back and confirm recovery.

```
NEW_IMAGE=gitlab-registry.endava.com/claudiu.sonel/petclinic-infralab:$TAG \
BASE_URL=http://localhost:8080 \
./scripts/deploy-with-gates.sh
```

A deliberate design choice: even after rollback succeeds, the script exits non-zero. This prevents automation from silently accepting a broken release. The environment is restored, but the deploy is still marked as failed.

Step 6 - Prove rollback works (test)

Rollback must be tested, not just implemented. The simplest deterministic test is to temporarily add a fake endpoint to the smoke endpoints list. That forces smoke failure without needing to rebuild a broken application.

- Add one non-existent endpoint to the smoke endpoints file (example: `/does-not-exist`).
- Run the deploy gate again; observe smoke failure, rollback, and a successful re-smoke.
- Remove the fake endpoint afterwards.

Running automatically post-deploy (GitLab CI/CD)

The phrase “run automatically post-deploy” means the smoke tests run in automation immediately after the deploy step, not as a manual command. Two pipeline patterns fit this requirement.

Pattern A - Single job gate

A single deploy job performs the deploy and then runs the deploy gate script. If smoke fails, rollback happens inside the same job, and the job fails.

- Deploy step: set image + wait for rollout.
- Post-deploy step: run `deploy-with-gates` (which includes smoke and rollback).
- Outcome: job fails if smoke fails, regardless of rollback success.

Pattern B - Separate validation stage

Deploy happens in one stage, smoke validation happens in the next. Rollback runs only if validation fails. This pattern requires storing the previous image value somewhere (artifact or a pre-deploy capture).

- Deploy job stores “previous image” as an artifact before changing it.
- Smoke job runs `smoke-test` against the deployed endpoint.
- Rollback job uses the stored “previous image” value and reruns smoke to confirm recovery.

CI security: store registry credentials and cluster auth in masked/protected variables; avoid printing them; keep logs clean.

Troubleshooting (fast patterns)

Use these checks before changing anything.

Is kubectl talking to the expected cluster?

```
kubectl config current-context  
kubectl get nodes
```

Did the Deployment update to the expected image?

```
kubectl -n petclinic get deploy petclinic -o jsonpath='{.spec.template.spec.containers[*].  
image}'
```

Why is a Pod failing?

```
kubectl -n petclinic get pods  
kubectl -n petclinic describe pod <pod-name>
```

Is the app reachable via port-forward?

```
kubectl -n petclinic port-forward svc/petclinic 8080:80  
curl -f http://localhost:8080/actuator/health/readiness
```

When something fails, avoid immediately rewriting scripts. First confirm whether the failure is networking, image pull/auth, or application readiness.

End-to-end runbook (repeatable checklist)

This is the shortest repeatable path that uses only the approach that worked in this implementation.

0) One-time: enable kubectl in WSL

```
export KUBECONFIG=/mnt/c/Users/<windows-user>/.kube/config  
# or copy to ~/.kube/config  
kubectl get nodes
```

1) Build + push (WSL)

```
cd <repo-root>  
TAG=smoke1  
podman login gitlab-registry.endava.com  
podman build --target runtime -t gitlab-registry.endava.com/claudiu.sonel/petclinic-infralab:$TAG .  
podman push gitlab-registry.endava.com/claudiu.sonel/petclinic-infralab:$TAG
```

2) Port-forward (leave running)

```
kubectl -n petclinic port-forward svc/petclinic 8080:80 # or 8080:8080
```

3) Run gate (deploy + smoke + rollback)

```
NEW_IMAGE=gitlab-registry.endava.com/claudiu.sonel/petclinic-infralab:$TAG \  
BASE_URL=http://localhost:8080 \  
./scripts/deploy-with-gates.sh
```

4) Rollback test (once)

- Temporarily add one fake endpoint to smoke endpoints list to force a failure.
- Run the gate again; confirm rollback returns to previous image and health recovers.
- Remove the fake endpoint.

Attach the scripts (smoke-test, deploy-with-gates, endpoints list) as separate files when submitting work. This PDF intentionally avoids embedding their full source.